Parallel and Distributed Computing

Dr. Imran Ashraf

imran.ashraf@hitecuni.edu.pk

Code that can be parallelized

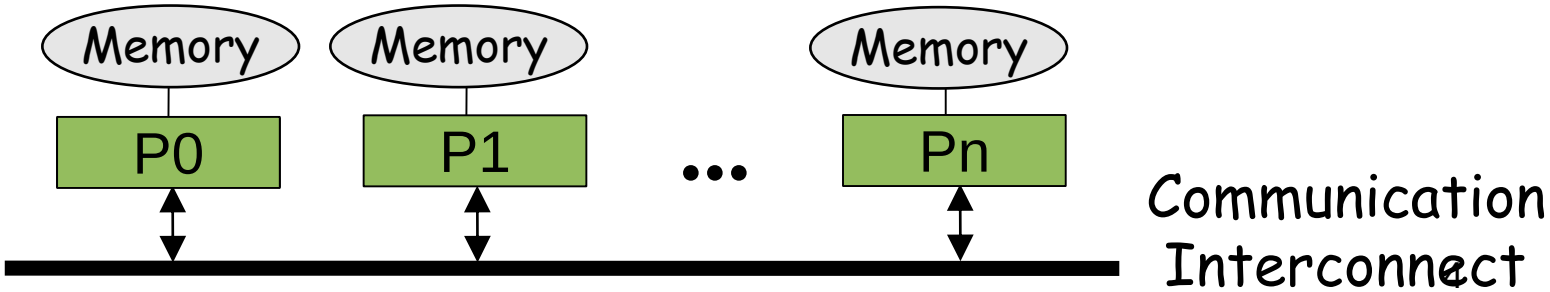
```
do i= 1 to max,
     a[i] = b[i] + c[i] * d[i]
end do
```

Parallel Computers

- Programming mode types
  - Shared memory
  - Message passing

Distributed Memory Architecture

- Each Processor has direct access only to its local memory
- Processors are connected via high-speed interconnect
- Data structures must be distributed
- Data exchange is done via explicit processor-to-processor communication: send/receive messages
- Programming Models
  - Widely used standard: MPI
  - Others: PVM, Express, P4, Chameleon, PARMACS, ...



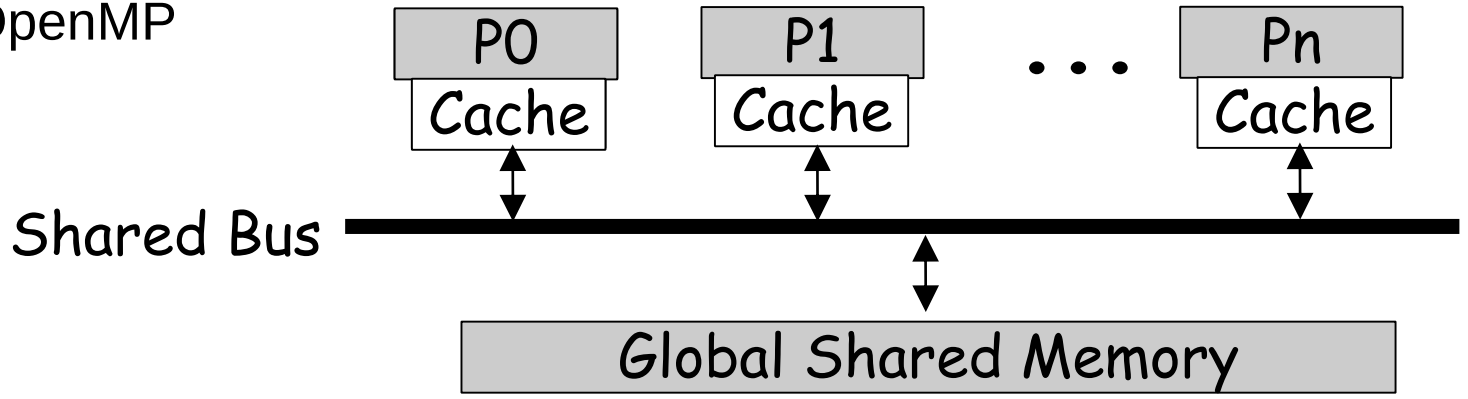
Message Passing Interface

MPI provides:

- Point-to-point communication
- Collective operations
  - Barrier synchronization
  - gather/scatter operations
  - Broadcast, reductions
- Different communication modes
  - Synchronous/asynchronous
  - Blocking/non-blocking
  - Buffered/unbuffered
- Predefined and derived datatypes
- Virtual topologies
- Parallel I/O (MPI 2)
- C/C++ and Fortran bindings
- *http://www.mpi-forum.org*

Shared Memory Architecture

- Processors have direct access to global memory and I/O through bus or fast switching network
- Cache Coherency Protocol guarantees consistency of memory and I/O accesses
- Each processor also has its own memory (cache)
- Data structures are shared in global address space
- Concurrent access to shared memory must be coordinated
- Programming Models
  - Multithreading (Thread Libraries)
  - OpenMP

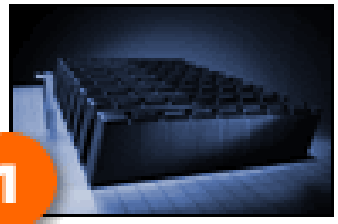


OpenMP

- OpenMP: portable shared memory parallelism
- Higher-level API for writing portable multithreaded applications
- Provides a set of compiler directives and library routines for parallel application programmers
- API bindings for Fortran, C, and C++

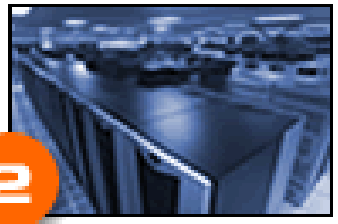
http://www.OpenMP.org

TOP 5 SUPERCOMPUTER SITES (November 2004)



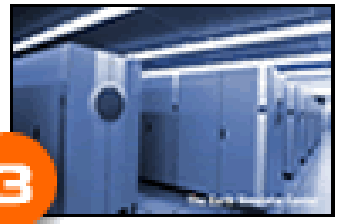
1

BlueGene/L
DOE/IBM
Rochester, USA
BlueGene/L DD2
Rmax: 70.72 TFlops



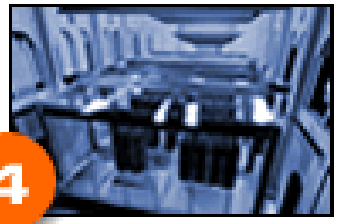
2

Columbia
NASA/Ames
Mountain View, USA
SGI Altix/Voltaire
Rmax: 51.87 TFlops



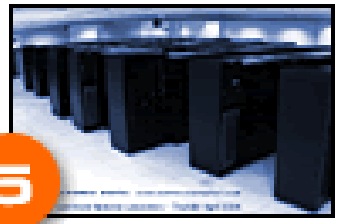
3

Earth Simulator
Earth Simulator Center
Yokohama
NEC
Rmax: 35.86 TFlops



4

MareNostrum
Barcelona Supercomputer Center
Barcelona, Spain
eServer BladeCenter JS20/Myrinet
Rmax: 20.53 TFlops



5

Thunder
Lawrence Livermore National Lab
Livermore, USA
Intel Itanium2 Tiger4/Quadrics
Rmax: 19.94 TFlops

Approaches

- Parallel Algorithms
- Parallel Language
- Message passing (low-level)
- Parallelizing compilers

Parallel Languages

- CSP - Hoare's notation for parallelism as a network of sequential processes exchanging messages.
- Occam - Real language based on CSP. Used for the transputer, in Europe.

More parallel languages

- ZPL - array-based language at UW. Compiles into C code (highly portable).
- C* - C extended for parallelism

Object-Oriented

- Concurrent Smalltalk
- Threads in Java, Ada, thread libraries for use in C/C++
  - This uses a library of parallel routines

Functional

- NESL, Multiplisp
- Id & Sisal (more dataflow)

Parallelizing Compilers

Automatically transform a sequential program into a parallel program.

1. Identify loops whose iterations can be executed in parallel.
2. Often done in stages.

Q: Which loops can be run in parallel?

Q: How should we distribute the work/data?

Data Dependences

Flow dependence - RAW. Read-After-Write. A "true" dependence. Read a value after it has been written into a variable.

Anti-dependence - WAR. Write-After-Read. Write a new value into a variable after the old value has been read.

Output dependence - WAW. Write-After-Write. Write a new value into a variable and then later on write another value into the same variable.

Example

1: A = 90;

2: B = A;

3: C = A + D

4: A = 5;

Dependencies

A parallelizing compiler must identify loops that do not have dependences BETWEEN ITERATIONS of the loop.

Example:

```
do I = 1, 1000
   A(I) = B(I) + C(I)
   D(I) = A(I)
end do
```

Example

```
Fork one thread for each processor
Each thread executes the loop:
  do I = my_lo, my_hi
          A(I) = B(I) + C(I)
          D(I) = A(I)
   end do
Wait for all threads to finish before
  proceeding.
```

Another Example

```
do I = 1, 1000
   A(I) = B(I) + C(I)
   D(I) = A(I+1)
end do
```

Yet Another Example

```
do I = 1, 1000
   A( X(I) ) = B(I) + C(I)
   D(I) = A( X(I) )
end do
```

Parallel Compilers

- Two concerns:
- Parallelizing code
  - Compiler will move code around to uncover parallel operations
- Data locality
  - If a parallel operation has to get data from another processor's memory, that's bad